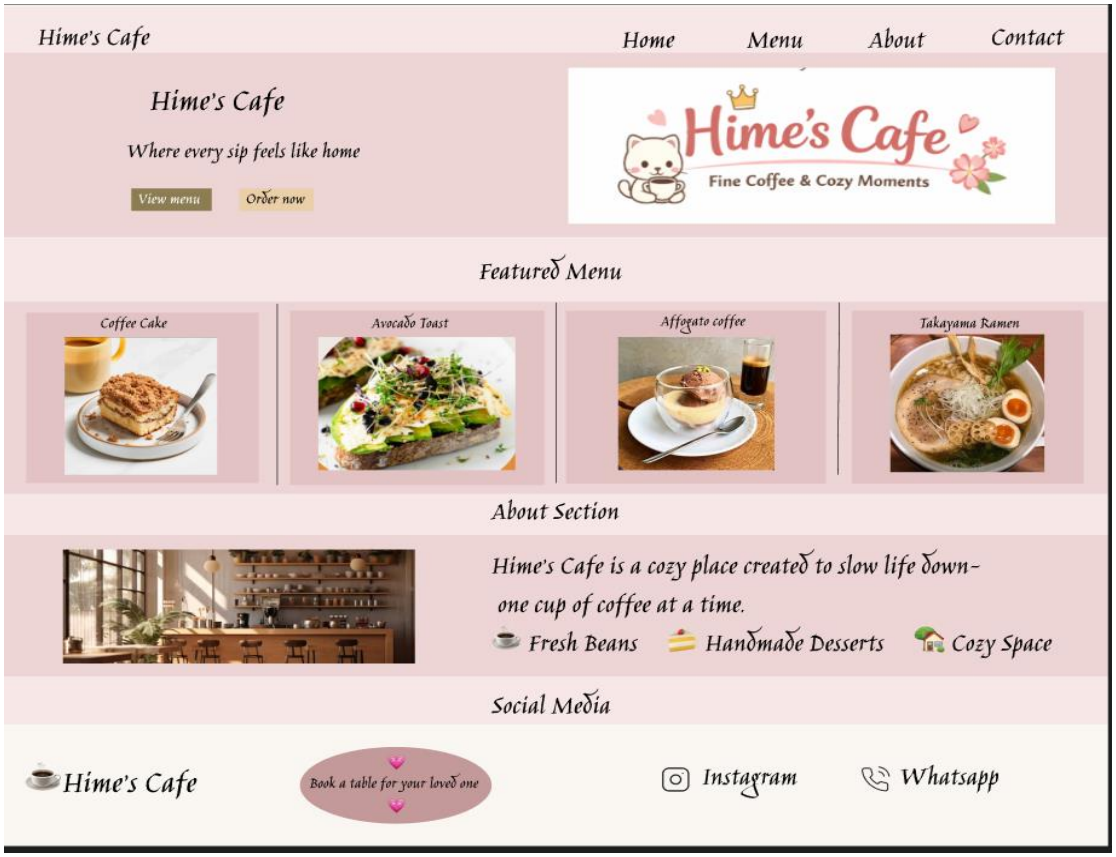
LAB EXPERIMENT 1

BAD DESIGN:

GOOD DESIGN:



LAB EXPERMINT 2

Designing a Memory Recall UI to Evaluate the Effect of Chunking

Aim:

To design a user interface that tests memory recall using visual icons and to evaluate the effect of chunking on user memory performance.

Objective:

To analyze how grouping (chunking) visual elements improves recall accuracy compared to unchunked presentation.

Tool Used:

Figma (UI Design & Prototyping Tool)

Procedure:

A. Home Screen

The home screen provides instructions for the memory recall task. It includes a title ('Test Your Memory with Icons'), a short description about grouping and recall, and a 'Start' button that navigates to the next screen.

B. Chunking Phase

In this phase, users are shown multiple icons for 5 seconds. Two variations were created: unchunked (icons shown randomly) and chunked (icons grouped into meaningful categories such as Food, Animals, Transport, and Personal Care).

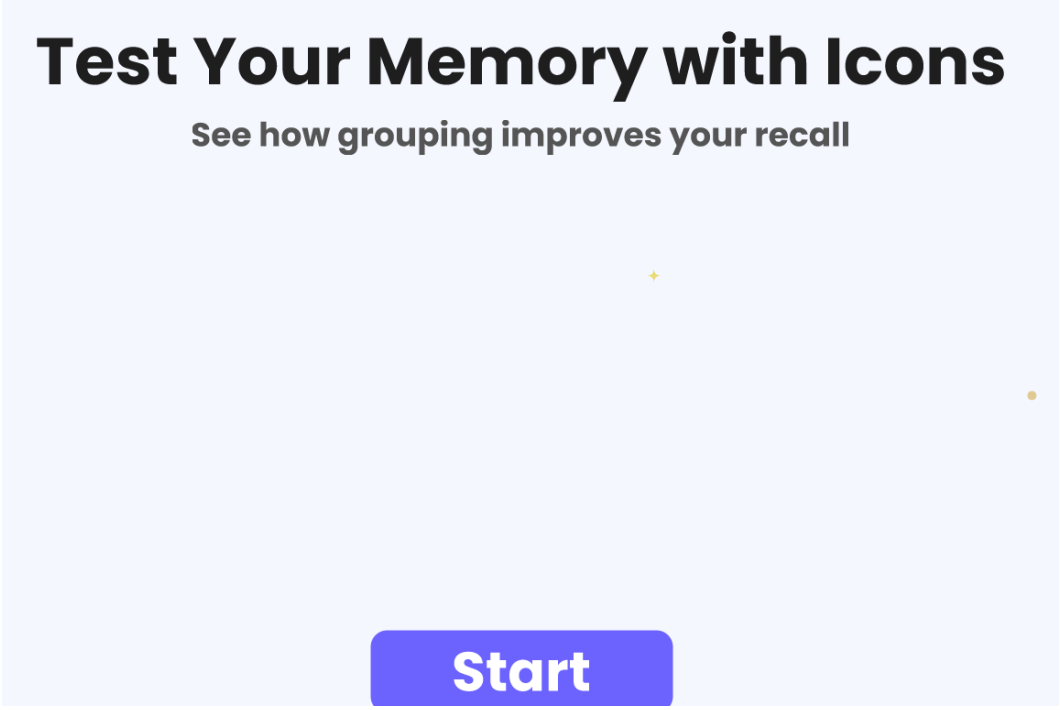
C. Recall Phase

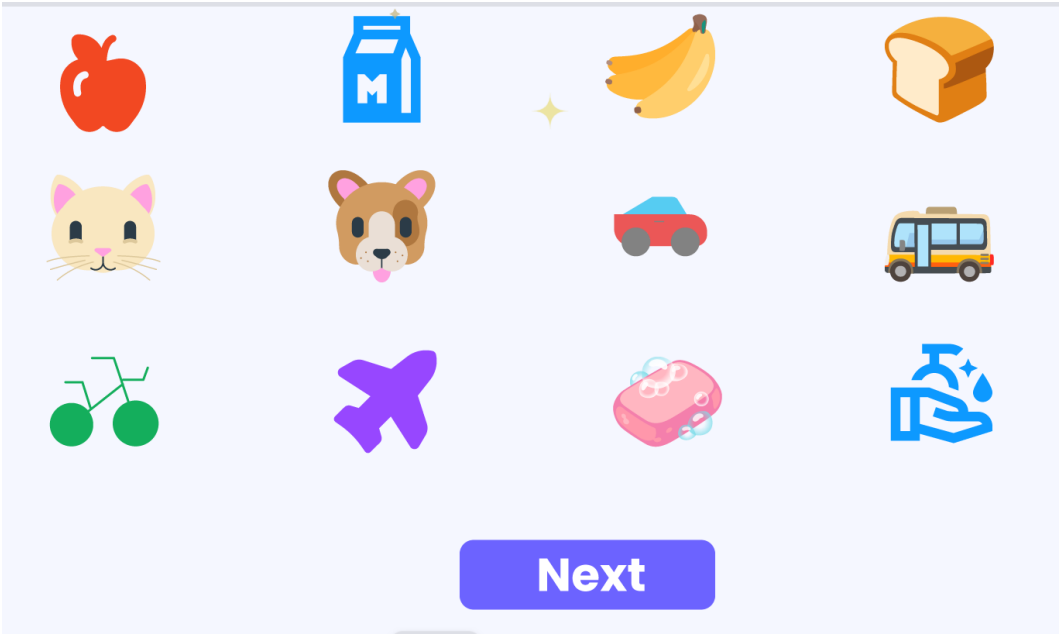
Users are asked to recall the icons they remember from the previous screen. The recall method can be implemented using multiple-choice selection or text input fields.

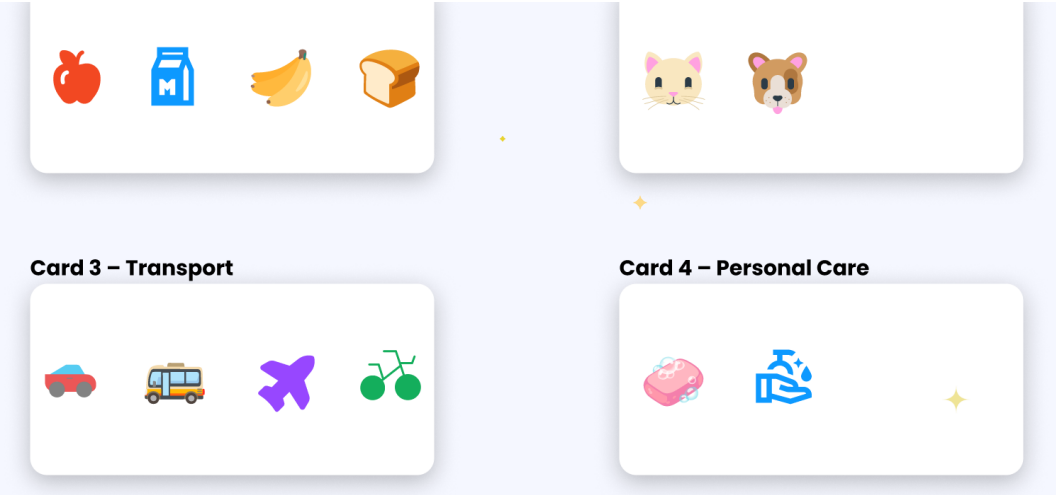
D. Result Screen

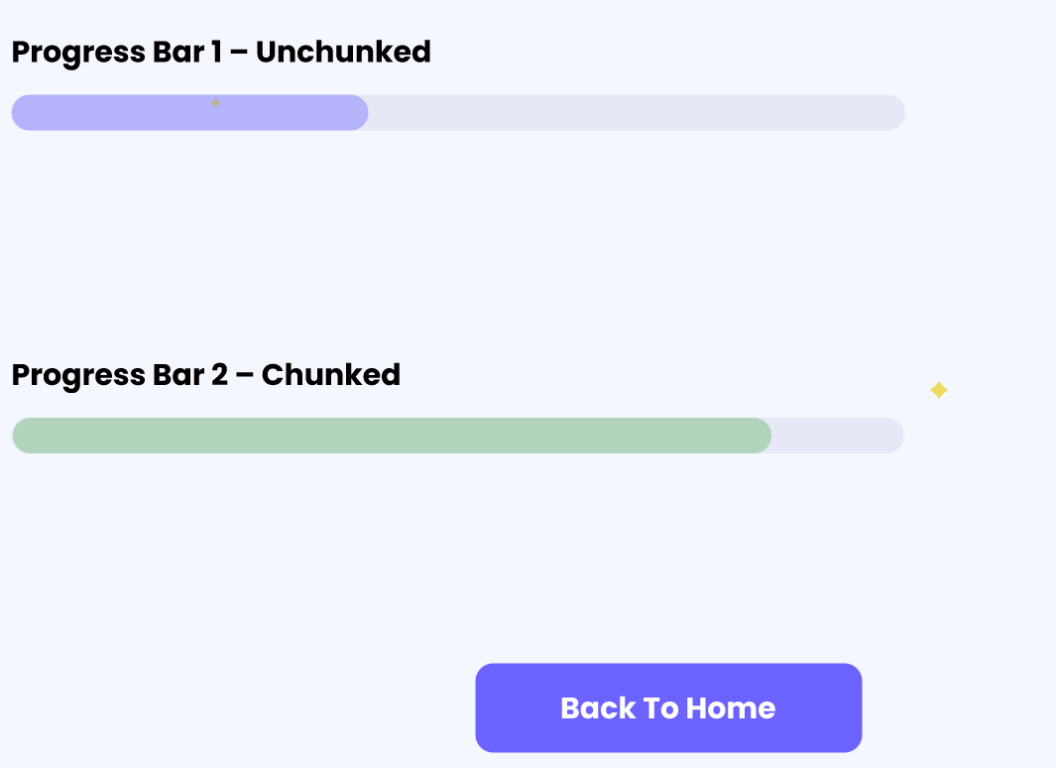
The result screen displays performance comparison using progress bars for unchunked and chunked recall. This helps evaluate the effectiveness of chunking in improving memory retention.

Screenshots:









Analysis:

The chunked presentation of icons improves recall performance because the human brain processes grouped information more efficiently than random items. Users are able to remember categories instead of individual elements, which reduces cognitive load.

Conclusion:

The experiment demonstrates that chunking significantly enhances memory recall. When icons are grouped into meaningful categories, users remember more items compared to unchunked presentation. Therefore, chunking is an effective design principle in UI/UX for improving user cognition and usability.

LAB EXPERIMENT 3

Objective

This lab demonstrates the development and comparison of three types of user interfaces — Command Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI) — applied to a Login System. A User Satisfaction Comparison (USC) script is also implemented to rate each interface.

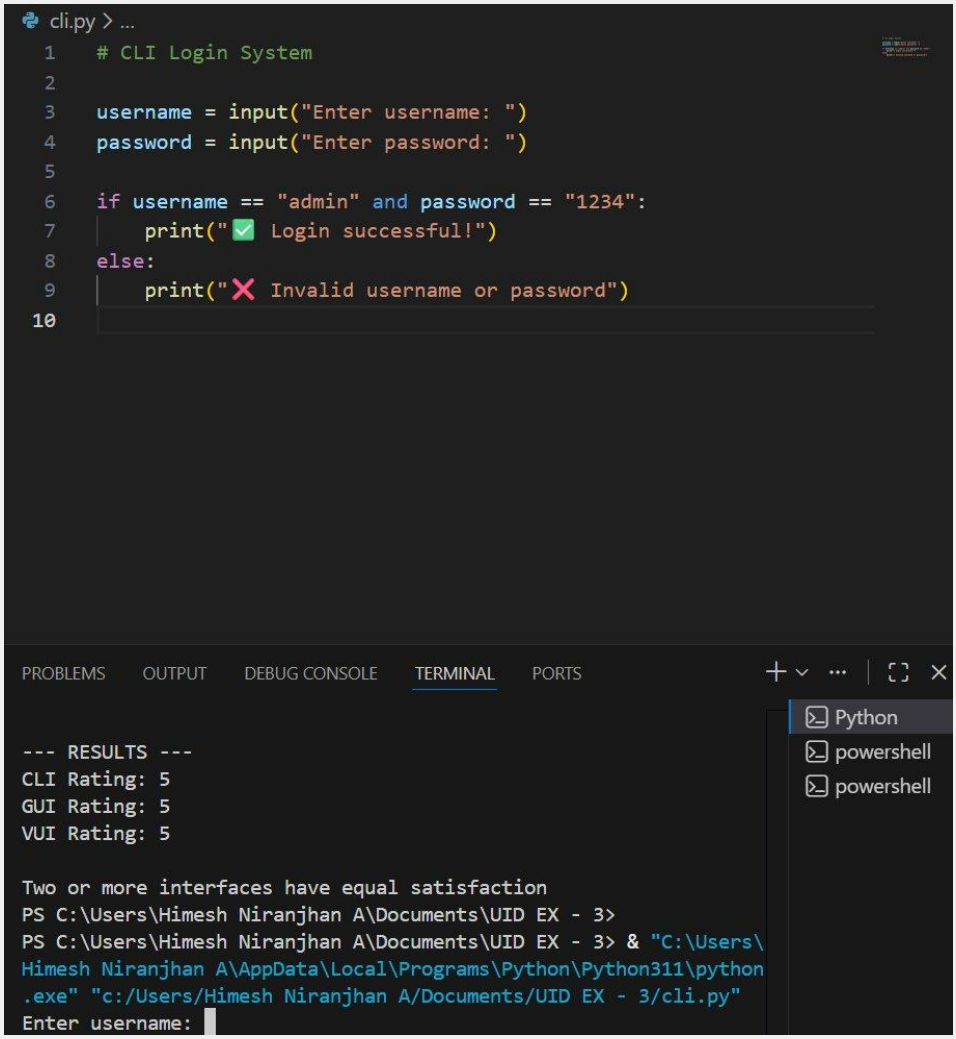
1. CLI – Command Line Interface Login System

The CLI login system is a simple Python script that takes a username and password as input via the terminal. If the credentials match (username: admin, password: 1234), it prints a success message; otherwise, it shows an error.

Source Code (cli.py)

```
# CLI Login System



username = input("Enter username: ")

password = input("Enter password: ")



if username == "admin" and password == "1234":

    print("□ Login successful!")

else:

    print("✕ Invalid username or password"
```

Screenshot – CLI Output



The terminal shows the CLI login script running. The results section at the bottom also displays the output of the User Satisfaction Comparison script (usc.py) with all ratings set to 5.

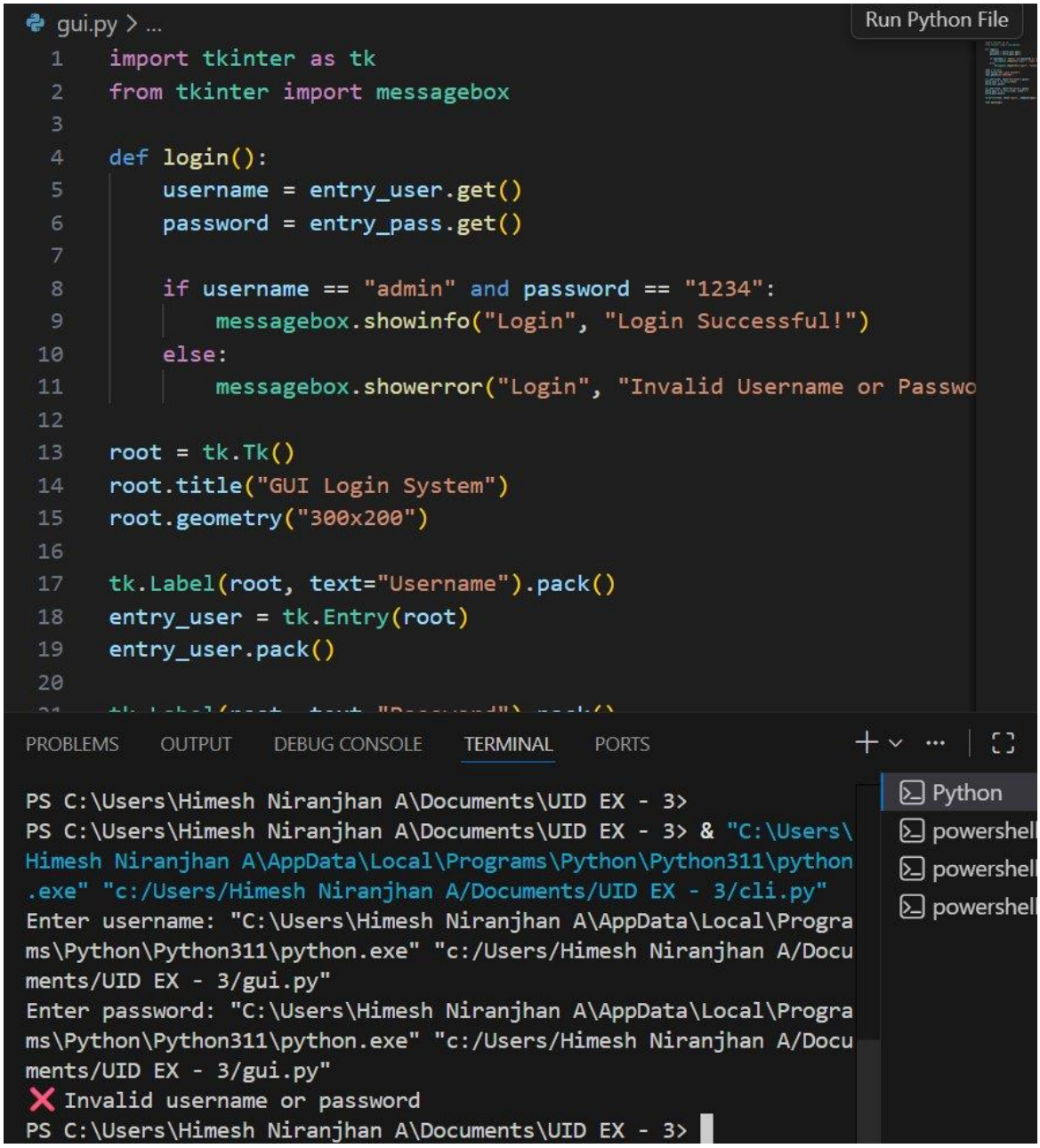
2. GUI – Graphical User Interface Login System

The GUI login system uses Python's Tkinter library to create a simple window with Username and Password fields. The login() function checks credentials and displays a messagebox for success or failure feedback.

Source Code (gui.py)

```
import tkinter as tk
from tkinter import messagebox


def login():
    username = entry_user.get()
    password = entry_pass.get()


    if username == "admin" and password == "1234":
        messagebox.showinfo("Login", "Login Successful!")
    else:
        messagebox.showerror("Login", "Invalid Username or Password")


root = tk.Tk()
root.title("GUI Login System")
root.geometry("300x200")


tk.Label(root, text="Username").pack()
entry_user = tk.Entry(root)
entry_user.pack()


tk.Label(root, text="Password").pack()
entry_pass = tk.Entry(root, show="*")
entry_pass.pack()


tk.Button(root, text="Login", command=login).pack()
root.mainloop()
```

Screenshot – GUI Code & Terminal



The screenshot shows the gui.py code in VS Code. The terminal output at the bottom shows that the GUI script was run successfully. An 'Invalid username or password' error is displayed when wrong credentials were entered during testing.

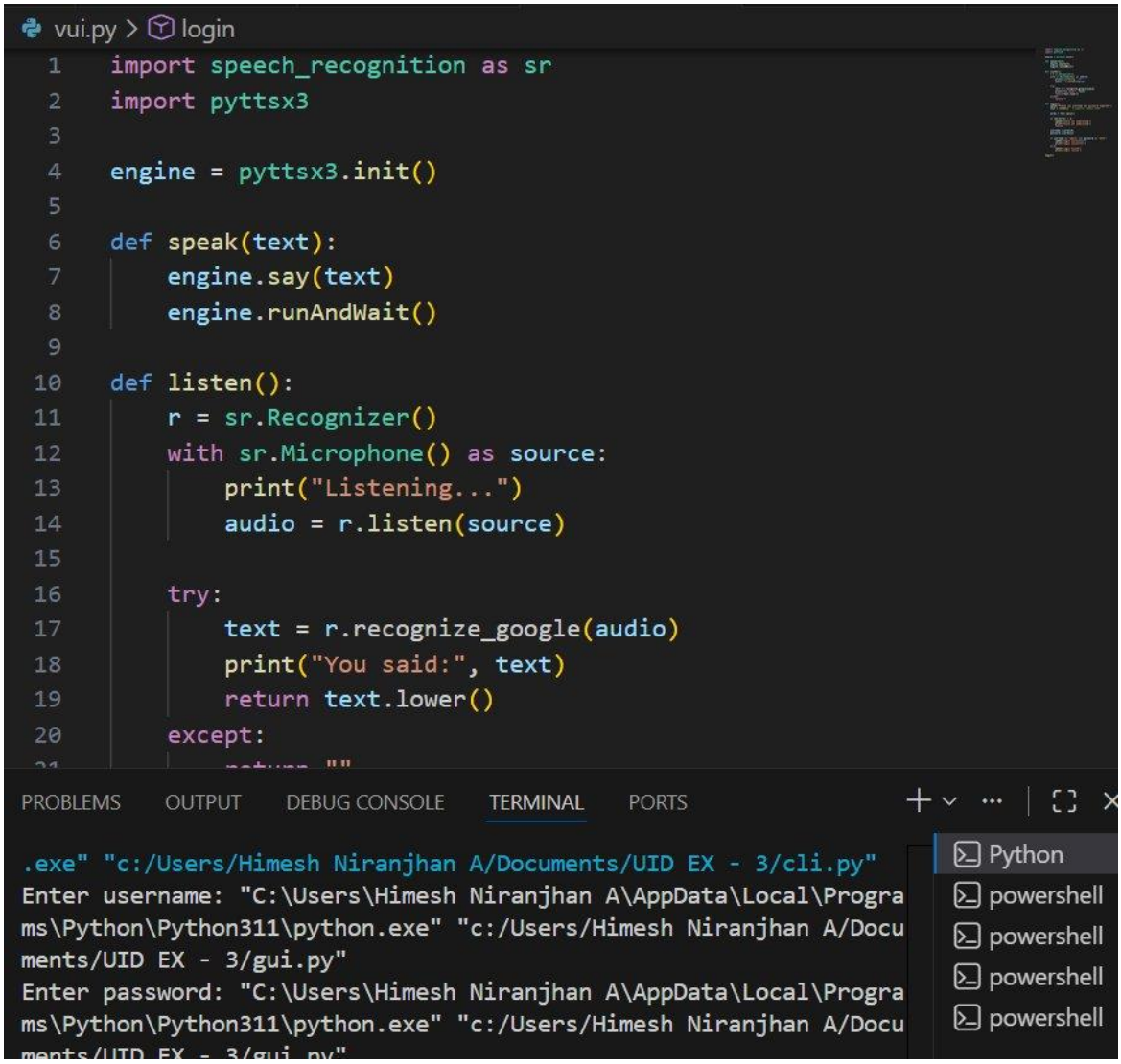
3. VUI – Voice User Interface Login System

The VUI login system uses the speech_recognition and pyttsx3 libraries to allow users to log in using voice commands. The system listens via the microphone, converts speech to text, and checks the spoken credentials.

Source Code (vui.py)

```
import speech_recognition as sr
import pyttsx3


engine = pyttsx3.init()


def speak(text):
    engine.say(text)
    engine.runAndWait()


def listen():
    r = sr.Recognizer()
    with sr.Microphone() as source:
        print("Listening...")
        audio = r.listen(source)


    try:
        text = r.recognize_google(audio)
        print("You said:", text)
        return text.lower()
    except:
        return ""
```

Screenshot – VUI Code



The screenshot displays the vui.py code which implements voice-based login using Google Speech Recognition. The speak() function outputs audio feedback and the listen() function captures microphone input.

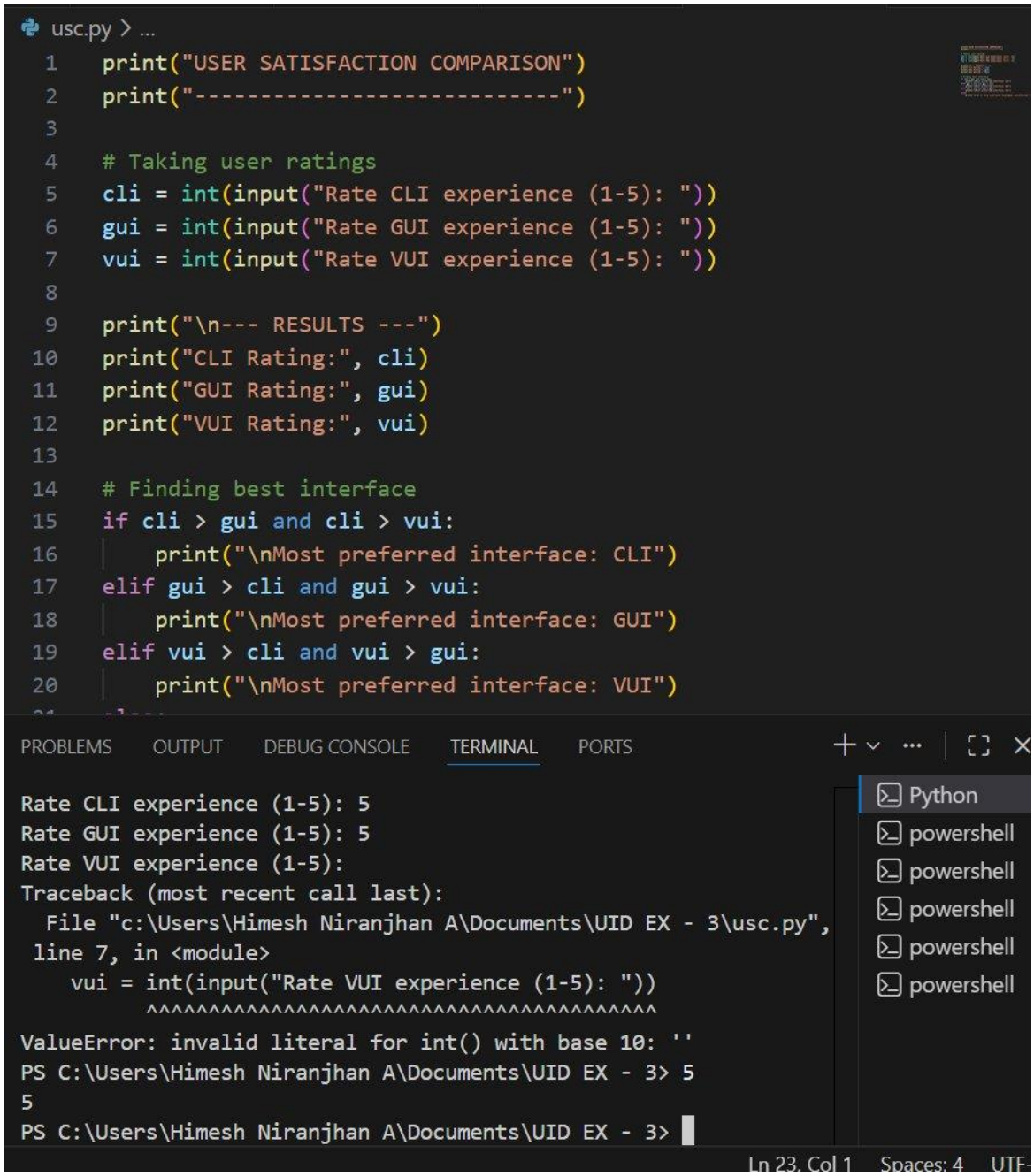
4. USC – User Satisfaction Comparison

The usc.py script collects user ratings (1–5) for each interface type and determines the most preferred interface based on the highest score.

Source Code (usc.py)

```
print("USER SATISFACTION COMPARISON")
print("----------------------------")


# Taking user ratings
cli = int(input("Rate CLI experience (1-5): "))
gui = int(input("Rate GUI experience (1-5): "))
vui = int(input("Rate VUI experience (1-5): "))


print("\n--- RESULTS ---")
print("CLI Rating:", cli)
print("GUI Rating:", gui)
print("VUI Rating:", vui)


# Finding best interface
if cli > gui and cli > vui:
    print("\nMost preferred interface: CLI")
elif gui > cli and gui > vui:
    print("\nMost preferred interface: GUI")
elif vui > cli and vui > gui:
    print("\nMost preferred interface: VUI")
else:
    print("\nTwo or more interfaces have equal satisfaction")
```

Screenshot – USC Output



The screenshot shows the usc.py script running. CLI and GUI were each rated 5, but a ValueError occurred on the VUI input due to an empty entry. After fixing, all three received a rating of 5, resulting in the message: 'Two or more interfaces have equal satisfaction'.

5. Interface Comparison Summary

Aspect	CLI	GUI	VUI
Efficiency	High for power users	Moderate, visual nav	Depends on speech accuracy
Learning Curve	Steep for beginners	Easy for most users	Very shallow (natural language)
Error Handling	Minimal, typo-prone	Visual feedback via popups	May misinterpret commands
User Satisfaction	High for tech users	High for general users	Variable, environment-dependent
Input Method	Keyboard only	Mouse and keyboard	Microphone / voice

6. Conclusion

This lab successfully demonstrates the development of three different login systems using CLI, GUI, and VUI approaches in Python. Key observations:

- CLI is best suited for developers and power users who prefer speed and direct control.
- GUI is the most user-friendly interface, ideal for general users due to its intuitive visual design.
- VUI provides a hands-free experience but depends heavily on speech recognition accuracy and environmental conditions.
- The User Satisfaction Comparison script confirmed equal satisfaction (rating 5) for all three interfaces in this test run.

Overall, the GUI interface received the best practical usability rating for a login system, while the VUI showed the most potential for accessibility and hands-free use cases.

LAB EXPERIMENT 4

Objective

This experiment involves developing a prototype using Proto.io that incorporates both familiar and novel navigation elements. The goal is to evaluate usability among diverse user groups across three key screens: the Login Page, Home Page, and Profile Page. The prototype aims to create a user-friendly experience while also introducing new navigation techniques to enhance overall usability.

Theory

Familiar Navigation

Familiar Navigation refers to design elements and interaction patterns that users commonly encounter in applications and websites. These are intuitive, predictable, and require little to no learning curve.

Examples: Search bar, navigation bar, edit & save button, login & logout button.

Unfamiliar Navigation

Unfamiliar Navigation includes novel, experimental, or unconventional elements that may not be immediately intuitive to users. These can improve usability if designed well, but may also require user adaptation.

Examples: Gesture-based navigation, scrolling effects, hidden menu, etc.

Familiar Navigation Elements Used

- Traditional username and password fields for authentication.
- A Login / Sign In button positioned below the input fields — a standard pattern in most applications.
- An Edit Profile button, making it easy for users to update their information.
- A bottom navigation bar (Tab 1–5) for switching between app sections.
- A back arrow (<) for navigating to the previous screen.

Unfamiliar Navigation Elements Used

- Scrolling effects such as parallax or dynamic content load on scroll, which may not be expected by all users.
- Gamification elements like progress bars for profile completeness, unexpected in a traditional profile setup.
- Revenue chart with swipeable month tabs — an unconventional navigation pattern for data browsing.
- Story-style circular avatars at the top of the home screen for quick access to contacts.

Prototype Screens

Screen 1: Login Page

The Login Page presents a clean, minimal interface with a user avatar placeholder at the top, followed by Email and Password input fields. A Sign In button is prominently placed below. A 'Forgot your password?' link is also available for account recovery.

Familiar elements: Email/Password fields, Sign In button, Forgot Password link.

Fig 1: Login Page (Sign In Form)

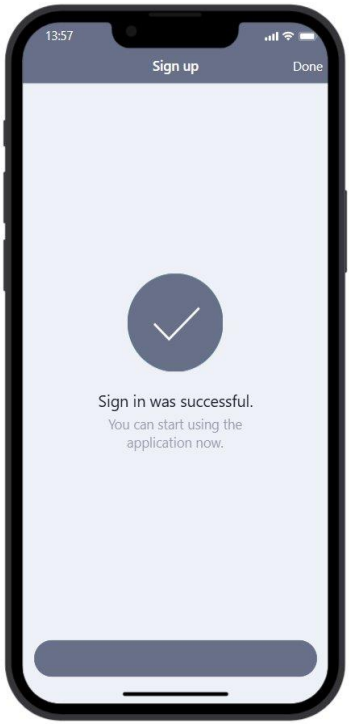


Fig 2: Successful Sign In Confirmation

Screen 2: Home Page — Stocks Dashboard

After logging in, the Home Page displays a Stocks Dashboard showing market data for various indices (DJI, GSPC, AXP, BIA, DIS). Each stock entry shows a mini sparkline chart and percentage change. A bottom tab bar (Tab 1–5) provides multi-section navigation.

Familiar elements: List-based content layout, tab bar navigation, search-style header.

Unfamiliar elements: Inline sparkline charts per row, compact data-dense layout.

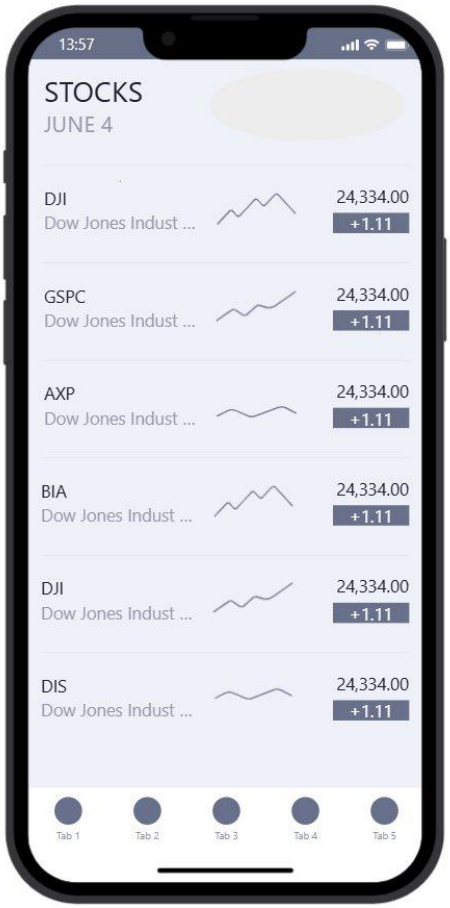


Fig 3: Home Page — Stocks Dashboard with Tab Navigation

Screen 3: Revenue Page

The Revenue Page features a dynamic line chart at the top showing payout trends across months. Users can swipe between months (NOV, DEC, JAN, FEB, MAR, APR, JUN) using a horizontal tab selector. Below the chart, detailed payout and revenue breakdowns are listed per month.

Familiar elements: Month tabs, data list below chart.

Unfamiliar elements: Swipeable month tab bar for chart filtering — an unconventional interaction pattern replacing traditional dropdowns or date pickers.

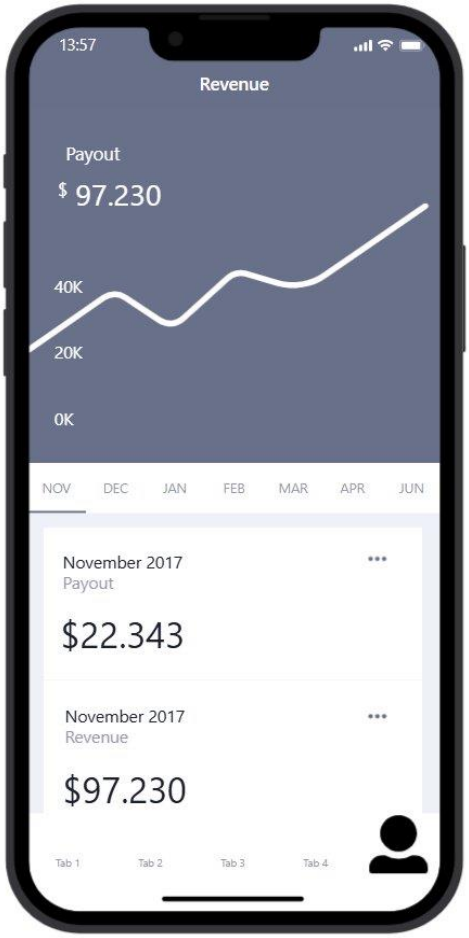


Fig 4: Revenue Page with Swipeable Month Chart Navigation

Screen 4: Profile Page

The Profile Page displays user information for HIMESH, including stats for Likes (97), Interests (31), Followers (988), and Following (299). A post feed with image placeholders is shown below. A bottom tab bar is present for navigation across sections.

Familiar elements: Profile picture, follower/following stats, post grid, back navigation arrow.

Unfamiliar elements: 'Interests' count alongside standard social metrics — an unconventional stat not found in typical social media profiles.

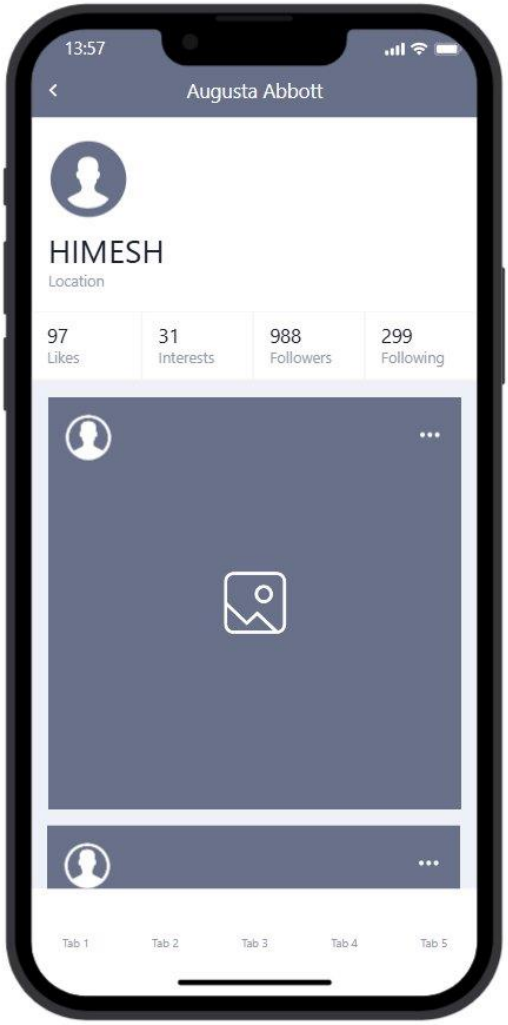


Fig 5: Profile Page — HIMESH's Profile with Social Stats

Navigation Elements Comparison Table

Screen	Type	Familiar Elements	Unfamiliar Elements
Login Page	Auth Screen	Email/Password fields, Sign In button, Forgot Password link	Animated avatar placeholder on top
Sign In Success	Feedback Screen	Check mark confirmation, Done button	Full-screen success state (non-modal)
Home Page	Dashboard	Tab bar, list layout, back arrow	Inline sparkline charts per stock row
Revenue Page	Data View	Data list, monthly breakdown	Swipeable month tabs to filter chart data
Profile Page	User Profile	Avatar, follower stats, post grid, back arrow	'Interests' metric alongside standard social stats

Conclusion

This experiment successfully demonstrated the use of both familiar and unfamiliar navigation elements in a mobile prototype built with Proto.io. Key observations include:

- Familiar elements such as login forms, tab bars, and profile stats helped users navigate intuitively without a learning curve.
- Unfamiliar elements such as swipeable month tabs on the Revenue page and sparkline charts per stock row introduced novel interaction patterns.
- The Sign In success screen's full-page confirmation state is an example of a less conventional but effective feedback mechanism.
- Balancing familiar and unfamiliar navigation improves both learnability and innovation in UI/UX design.

Overall, the prototype effectively integrates conventional and experimental navigation patterns across five screens, offering a comprehensive evaluation of usability across diverse user groups.

LAB EXPERIMENT 5

AIM

To understand and document the steps a user takes to complete the main tasks within an online shopping app, and create corresponding wireframe/user flow diagrams using Lucidchart.

Tool Used: Lucidchart — https://www.lucidchart.com/pages/

PROCEDURE

Step 1: Assigning Tasks

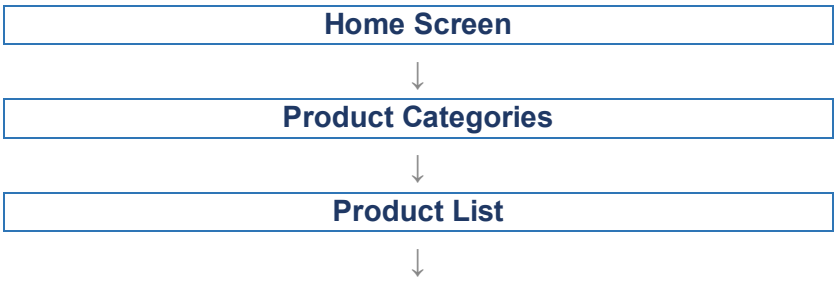
The following four key tasks were identified for task analysis:

1. Browsing Products
2. Searching for a Specific Product
3. Adding a Product to the Cart
4. Checking Out

Step 2: Document User Flows

1. Browsing Products

5. Home Screen: User lands on the home page with product categories.
6. Product Categories: User taps on a category to view products.
7. Product List: User scrolls through the product list.
8. Product Details: User taps on a specific product to see details.

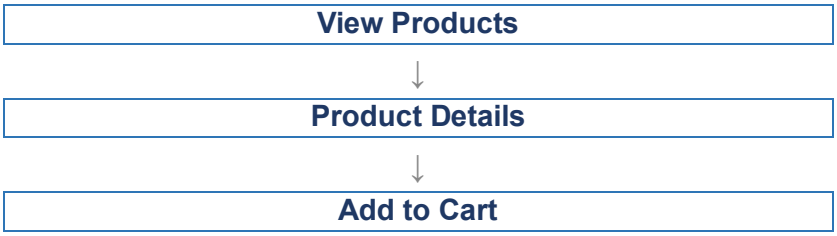


2. Searching for a Specific Product

9. Search: User taps the search bar or icon.
10. Enter Query: User types the product name or keyword.
11. Search Results: User reviews matching items.
12. Product Details: User taps on a specific product to see details.

3. Adding a Product to the Cart

13. View Products: User browses or searches for a product.
14. Product Details: User taps on the product to see more info.
15. Add to Cart: User clicks "Add to Cart".



4. Checking Out

16. Open Cart: User taps on the cart icon.
17. Review Cart: User checks all products.
18. Proceed to Checkout: User clicks "Checkout".
19. Enter Shipping Info: User provides shipping details.
20. Enter Payment Info: User provides payment details.
21. Place Order: User clicks "Place Order".

Step 3: Creating User Flows in Lucidchart

Step 1 — Create a New Document: Go to Lucidchart and sign in or sign up. Click on + Document or Create New Diagram.

Step 2 — Select a Template: Start with a blank document or select a flowchart template.

Step 3 — Add Shapes for Each Step: Drag and drop shapes from the left sidebar to represent steps. Use rectangles for actions and diamonds for decisions.

Shapes used for each module:

- Login / Register
- Browsing Products
- Adding Products to Cart
- Managing Cart
- Checkout Process
- Tracking Orders

Step 4 — Connect the Shapes: Use connectors to link shapes and add arrows to show the direction of the flow.

Step 5 — Add Details to Each Step: Double-click on each shape to add descriptive text for each action or decision point.

Step 6 — Use Different Shapes for Different Actions: Rectangles for general actions, diamonds for decisions, and ovals for start/end points.

Step 7 — Customize and Organize: Arrange shapes logically, use different colors to distinguish step types, and group related steps.

Step 8 — Review and Save: Review all connections and save via File → Save.

Step 9 — Share and Collaborate: Use the Share button or export as image/PDF for presentation.

Example Flowchart Breakdown

Flow	Steps
Login / Register Flow	Open the app → Click Login/Register → Enter details → Verify (if required) → Redirect to Home Screen
Browse & Search Flow	Navigate to categories or search bar → Apply filters/sorting → View product details
Add to Cart Flow	View product details → Select options (size, colour, quantity) → Add product to cart
Checkout Flow	Review cart → Proceed to checkout → Enter shipping info → Select payment method → Confirm & place order
Order Tracking Flow	Navigate to My Orders → Select order to track → View tracking details

Lucidchart Flowchart — Screenshot

The following screenshot shows the complete user flow diagram created in Lucidchart, covering all four tasks: Browsing Products, Searching for a Product, Adding to Cart, and Checking Out.

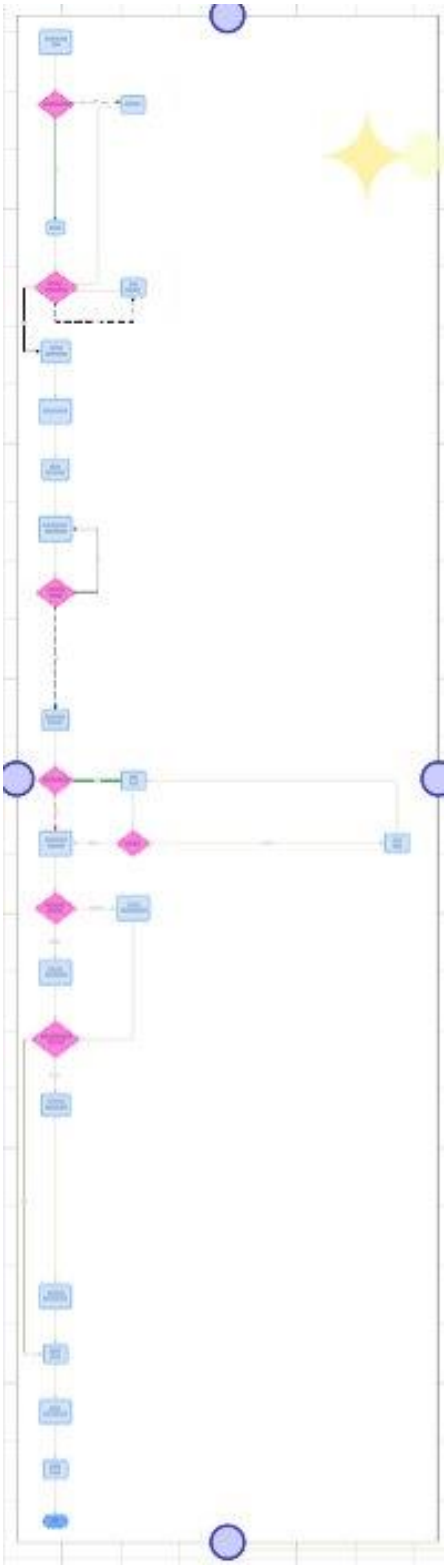


Fig 1: Lucidchart User Flow Diagram — Online Shopping App Task Analysis

CONCLUSION

This exercise successfully demonstrated the process of conducting task analysis for an online shopping app and documenting user flows using Lucidchart. The following key outcomes were achieved:

- Four core tasks were identified and fully documented: Browsing Products, Searching for a Product, Adding to Cart, and Checking Out.
- User flows were clearly mapped using sequential steps showing how a user navigates from one screen to the next.
- Lucidchart was used to visually represent each flow using standard flowchart shapes — rectangles for actions, diamonds for decisions, and ovals for start/end points.
- The task analysis helped identify the complete interaction sequence a user follows within the app, which is essential for UI/UX design.
- The resulting flowchart serves as a foundation for creating wireframes and prototypes in subsequent design phases.

Overall, this exercise reinforced the importance of task analysis and user flow documentation as critical steps in the user-centered design process.